

CloseCut

Engineering Architecture

The Canonical Technical Specification

Version 1.0

Public Reading Edition

Public Reading Edition

The Canonical Technical Specification Version 1.0 · 13 July 2026

Document Status

This public edition explains the engineering principles, application architecture, data boundaries, local-first model, feature modularization, testing philosophy, cross-platform direction, and architecture governance behind CloseCut. It intentionally omits implementation-sensitive backend, security-rule, environment, monitoring, deployment, and release-operation details.

Purpose

CloseCut is a private, memory-centered film and series journal with local-first persistence, thoughtful decision support, and deliberately bounded social sharing. This edition explains the durable architecture principles that support that product intent without exposing operational details that belong in private engineering documentation.

Audience

The primary audience is mobile and product engineers, technical designers, architecture reviewers, recruiters, collaborators, and future maintainers who want to understand the system at a principled level.

Relationship with the Canonical Library

Product Vision defines what CloseCut should become. Experience Design defines how it should feel and behave. This Engineering Architecture edition explains how the application can support both while preserving privacy, local-first trust, accessibility, maintainability, and cross-platform consistency.

Public Edition Scope

Included: engineering philosophy, architecture principles, application layers, state and data boundaries, local-first persistence, synchronization concepts, feature architecture, testing, performance, accessibility engineering, Android parity, future architecture, and governance.

Excluded: exact backend implementation, security rules, index definitions, operational monitoring details, environment configuration, CI/CD internals, release checklists, audits, technical-debt reports, and open internal decisions.

1. Engineering Philosophy

CloseCut treats privacy, continuity, and emotional safety as system qualities. The architecture therefore optimizes for durable personal ownership before network convenience. A journal entry must remain usable without a network, a recommendation must be explainable without opaque profiling, and social behavior must remain explicitly scoped to trusted Circles.

The governing sequence is why before how. Product intent determines invariants; experience intent determines interaction constraints; engineering selects mechanisms that satisfy both. The system rejects architecture theater: a layer exists only when it creates a stable ownership boundary, test seam, or

platform contract.

Current evidence supports a pragmatic clean architecture rather than a pure textbook implementation. Domain models, local models, Firestore DTOs, repositories, remote data sources, sync services, and feature views are distinct. Some views still instantiate collaborators and orchestrate effects. This is accepted as current state, not elevated into the target architecture.

2. Architecture Principles

1. Local usefulness is non-negotiable. Personal history and drafting must survive temporary network loss.
2. Privacy is enforced at every boundary. UI affordances, local ownership, Firestore rules, DTOs, and deletion flows must agree.
3. One semantic model, multiple storage representations. Domain concepts remain stable while SwiftData, Firestore, and Android Room use platform-appropriate persistence models.
4. Repositories own data policy. Views request outcomes; repositories decide local mutation, queueing, deduplication, and remote coordination.
5. Remote infrastructure is replaceable; product semantics are not. Firebase is an implementation choice behind explicit boundaries.
6. Synchronization is a state machine, not a side effect. Pending work, retries, conflicts, tombstones, and user-visible status are modeled.
7. Social is subordinate to personal memory. A Circle may receive a projection of an entry; it does not become the owner of that memory.
8. Accessibility and localization are release criteria. They are not post-launch polish.
9. Incremental modularization beats speculative frameworks. Introduce protocols where they protect high-value seams.
10. Documentation is part of the change. Architectural behavior is incomplete until its canonical description and decision record are updated.

3. System Overview

The iOS application is composed of an application shell, feature presentation, shared UI/design foundations, domain and persistence models, repositories, remote data sources, synchronization services, Firebase infrastructure, and external metadata integration.

At launch, `CloseCutApp` creates session-level services and the `SwiftData` model container. `AppDelegate` configures Firebase, Firestore persistence/index behavior, and notification delegation. `RootView` resolves authentication, session preparation, launch/onboarding gates, and the main tab experience.

The dominant data path is: View -> feature state/ViewModel -> Repository -> SwiftData mutation -> PendingAction queue -> Sync service -> RemoteDataSource -> Firestore. Pull synchronization reverses the remote portion and maps DTOs back into local records under conflict policy.

The supplied backend contains callable Cloud Functions for Circle invite workflows and account-deletion operations, Firestore and Storage rules, composite indexes, and emulator configuration. The deployed environment itself was not exported; repository configuration is authoritative for intended deployment, while actual production drift remains an operational verification item.

4. Technology Stack

iOS: Swift, SwiftUI, SwiftData, Combine where observable streams are needed, UIKit bridges for platform services, XCTest/XCUITest.

Backend: Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions for Firebase v2, TypeScript, Firebase Admin SDK, callable HTTPS functions.

External data: TMDB client, endpoint, configuration, media repository, DTOs, genre mapping, and image URL construction. TMDB supplies metadata; it does not own journal truth.

Build and configuration: Xcode project, `.xcconfig`-based configuration, entitlements, plist metadata, Firebase CLI project mapping, emulator suite, TypeScript compilation, ESLint, and npm lockfile.

Design: repository-native design foundations and components aligned to the separate Experience Design System.

The current import profile is led by SwiftUI (210), Foundation (199), SwiftData (62), FirebaseFirestore (20), UIKit (18), Combine (14), XCTest (9), UserNotifications (6).

5. Architectural Decisions

The binding decisions are captured in ADR-001 through ADR-008 and reproduced in /adr. Together they establish local-first authority, repository ownership, pragmatic MVVM, Firebase separation, privacy-first sync, feature modularization, explicit offline behavior, and cross-platform semantic parity. No single ADR permits bypassing another: for example, a feature cannot claim cross-platform parity by copying Firestore DTOs directly into UI state, because repository and privacy decisions still apply.

6. Module Boundaries

The current physical structure is primarily folder-based rather than separate Swift packages. That is acceptable while the codebase remains a single application target, provided dependency direction is enforced in review and tests.

Observed source distribution: Feature: 251, Shared UI: 30, Domain: 30, Other: 18, Remote Data Source / DTO: 18, Local Persistence: 12, Recommendation Engine: 11, Repository: 11, Service: 11, Application Shell: 10, Sync: 9, Test: 8, Infrastructure: 1.

Target boundaries:

- App owns composition, lifecycle, root routing, and process-wide integrations.
- Features own user flows, presentation state, and feature-specific components.
- Core Domain owns product semantics and pure policies.
- Core Repositories own aggregate access and write policy.
- Core Local owns SwiftData representations and local stores.
- Core Remote owns Firebase paths, DTOs, and transport behavior.
- Core Sync owns queues, reconciliation, retries, and conflict policy.
- Infrastructure owns platform and third-party adapters.
- Shared UI / Design System owns reusable visual primitives without feature data dependencies.

A future extraction into packages is justified only when build isolation, ownership, or reuse benefits exceed the cost of additional public APIs.

7. Dependency Rules

Allowed direction is inward toward stable semantics. App may depend on all composition-time modules. Features may depend on domain types, repository/service interfaces, and shared UI. Repositories may depend on domain, local persistence, remote data sources, and sync abstractions. Remote data sources may depend on Firebase; domain must not. Shared UI must not depend on feature repositories.

Direct Firebase imports in views are migration targets. Direct SwiftData access in views is acceptable for read-only projections when predicates enforce ownership and no business policy is embedded in presentation. Mutations should flow through repositories. Cycles between feature folders are prohibited; shared behavior must move to a stable shared or domain boundary.

8. App Lifecycle

Application startup has five architectural responsibilities: initialize infrastructure, construct persistence, establish identity, prepare session-local data, and resolve the first user-visible route. Failures must be classified by recoverability. Firebase bootstrap failure is fatal for authenticated remote behavior but should not corrupt local data. Session synchronization failure is recoverable and must preserve local access with a visible pending state.

Sign-out and account deletion are different lifecycle transitions. Sign-out terminates credentials and listeners while preserving locally scoped data only when product policy explicitly permits it. Account deletion must complete remote preflight, remote deletion, local purge, notification cleanup, cache cleanup, and credential termination in an idempotent sequence.

9. Navigation Architecture

`AppNavigationCoordinator`, `AppRoute`, and `AppTab` form the explicit navigation vocabulary. Root gates decide authentication/onboarding/main application state; feature navigation should use typed routes rather than string identifiers. Notification deep links are translated through a dedicated router before reaching feature destinations.

Navigation state must not become a second source of domain truth. Routes carry stable identifiers or lightweight launch context, then destination loaders obtain authoritative local data. Deep links must fail safely when content was deleted, belongs to another account, or is not yet synchronized.

10. Feature Modules

Current feature folders include Social (47), Entries (38), Home (32), AwardsCulture (24), Settings (23), Battle (19), QuickAdd (15), Wrap (12), Notifications (12), Watchlist (7), Discover (6), Onboarding (6), Insights (5), Search (3), AppShell (2). Each feature may contain views, components, focused ViewModels, pure helpers, and feature adapters. It may not define competing persistence truth.

A feature qualifies for extraction into a package when it has a stable domain boundary, independent test surface, and limited cross-feature imports. Circles and Watch Together are candidates because they have backend contracts and substantial substructure. Smaller features should remain folders until extraction improves rather than obscures ownership.

11. MVVM

CloseCut uses pragmatic MVVM. Views render observable state and emit user intent. ViewModels coordinate presentation-specific state, validation, cancellation, and repository calls. Repositories and services own persistence, network, and cross-feature policy.

A ViewModel must not become a universal service locator or duplicate local storage. Pure formatting may remain in view helpers; business decisions such as duplicate policy, recommendation scoring, membership authorization, and conflict resolution belong outside presentation. Long-lived ViewModels must be `@MainActor` where they mutate UI-observed state and must cancel tasks/listeners when their lifecycle ends.

12. Repository Layer

Repositories are the application-facing data authority. They translate between domain objects and local/remote representations, enforce ownership predicates, coordinate local writes and pending actions, and expose intention-revealing operations.

Current repositories cover entries, Circles, user profiles/state, watchlist, Watch Together plans, battle results, and QuickPick recommendations. The target is not protocol proliferation. Protocols are required where tests, platform substitution, emulator isolation, or multiple implementations create genuine value. Repository APIs should avoid leaking `ModelContext`, Firestore snapshots, or DTOs.

13. Services

Services represent capabilities that do not naturally belong to an aggregate repository: authentication, account deletion, local purge, profile-photo storage, metadata enrichment, notifications, calendar export, external links, and Circle invite behavior.

A service must define its failure model. User-cancelled, offline, unauthorized, validation, quota, server, and invariant failures are not interchangeable. Services performing irreversible operations require idempotency and explicit completion semantics.

14. Dependency Injection

The current application constructs several session-level dependencies in the app shell but also instantiates repositories and services inside feature views. The canonical target is a small composition root with scoped dependency containers: application scope, authenticated-session scope, and short-lived feature scope.

Use initializer injection for required collaborators and environment injection for genuinely shared UI/session dependencies. Avoid global mutable singletons. Firebase SDK singleton access may remain inside infrastructure adapters, but callers should receive CloseCut-owned interfaces. Test composition must support in-memory SwiftData, emulator-backed Firebase, deterministic clocks, and stub external metadata.

15. State Management

State is classified as domain state, persisted local state, remote synchronization state, ephemeral presentation state, and derived state. SwiftData is the durable local source for personal application state. Firestore is the remote synchronization and sharing source. View state owns transient selections, sheet presentation, and loading indicators. Derived state must be recomputable and should not be persisted unless computation cost or historical semantics justify it.

Every asynchronous state machine must expose idle, loading, success/empty, recoverable failure, and terminal failure when applicable. Stale data is preferable to an empty screen when the last known local state is trustworthy.

16. Error Handling

Errors are converted at boundaries. Firebase and URL-session errors must not reach views verbatim. Repositories map infrastructure failures into domain-relevant categories; presentation maps those categories into calm, actionable language.

Logging must retain technical context without recording private journal text, authentication tokens, invite codes, email addresses, or unrestricted identifiers. Retry is appropriate only for transient failures and must use bounded backoff. Validation failures are never retried automatically.

17. SwiftData

SwiftData is the current local persistence engine and the primary source for responsive UI. Models are registered centrally by `AppModelContainer`. Local entities include entries, user profile/state, Circles and memberships, watchlist items, Watch Together plans/responses, QuickPick recommendations, battle results, and pending actions.

Every user-owned entity must carry a stable owner identifier. Queries must predicate by active user whenever possible rather than fetching all rows and filtering in memory. Schema changes require explicit migration assessment, backup/rollback reasoning, and compatibility with pending actions created by older app versions.

18. Firestore

Firestore stores synchronized and shared representations. Collection paths are centralized in `FirestorePaths`; DTOs isolate transport shape from domain and `SwiftData` models. Rules and indexes are versioned beside the application.

Firestore is not the UI source of truth for personal flows. Remote listeners or pulls update local state through repositories/sync services. Shared social projections may require fresher remote behavior, but they must still map through owned boundaries. Collection-group queries and denormalized member identifiers are accepted where required for authorization and discovery, with explicit consistency maintenance.

19. Authentication

Firebase Authentication establishes remote identity. `AuthService`, `SessionViewModel`, and launch gates translate SDK state into CloseCut session state. Authentication does not by itself authorize data; Firestore and Storage rules validate ownership and Circle membership.

Identity changes require cancellation of listeners, reset of session-scoped services, local ownership isolation, and prevention of data bleed between accounts. Anonymous or alternative authentication modes may be added only with a documented merge and deletion policy.

20. User Models

The user aggregate separates authentication identity from profile and application state. Authentication owns UID and provider facts. User profile owns display-facing attributes and profile photo references. User state owns onboarding and application progress. Local and Firestore representations may differ, but stable IDs and timestamps must preserve merge behavior.

Email is not a durable foreign key. UID is the authority. Profile fields used inside Circle snapshots are denormalized for continuity and must be treated as display snapshots, not live identity truth.

21. Entry Models

Entry is the canonical memory aggregate. One entry represents one viewing memory, with media metadata, temporal context, optional reaction/rating/reflection, mood, rewatch semantics, cinema context, visibility, and Circle-sharing references. Quick Add and full journal flows create the same semantic aggregate at different completeness levels.

Local LocalEntry, remote FirestoreEntryDTO, and domain Entry are separate representations. Mappers must preserve stable identity, ownership, creation/update timestamps, deletion/tombstone state, and sharing intent. External TMDb metadata is a snapshot used to enrich memory; user-authored content remains authoritative.

22. Timeline Models

The timeline is a projection over entries, not a separate source of truth. Grouping, sorting, search, filters, and display sections are derived from locally owned entries. Persisting timeline-specific copies would create divergence and is prohibited.

Ordering uses viewing date as the primary memory chronology and stable tie-breakers for deterministic rendering. Unknown or rough dates must preserve user intent rather than fabricate precision.

23. Recommendation Models

QuickPick uses explicit candidates, signals, scores, no-repeat policy, rewatch rules, history analysis, and reason construction. The output is a recommendation plus an explanation code and state, not an opaque ranking feed. Local recommendation history may be persisted to avoid repetition and support continuity.

Recommendation inputs must be privacy-minimized and deterministic enough to test. Future ML may assist candidate scoring, but it may not remove explainability, transmit private journal text by default, or silently replace the local rules engine.

24. Sync Engine

Synchronization is built around `PendingAction`, queue helpers, per-aggregate sync services, DTOs, and conflict policy. The local write commits first, then a pending action records remote intent. Push processing is idempotent by stable entity/action identifiers. Pull processing maps remote state into local records and resolves conflicts without bypassing ownership.

A sync cycle must be observable: started, progress/aggregate results, completed, partially failed, or blocked by authentication/network. Queue records retain retry count and last failure category, not sensitive payload logs. Poison actions require quarantine or user-visible remediation rather than infinite retry.

25. Offline-first

Offline-first means more than Firestore caching. Core personal actions must complete against SwiftData, surface pending status, and synchronize later. Features that fundamentally require remote membership or fresh shared state must degrade honestly: they may show cached content and explain that changes will send later, but they must not imply server acceptance.

The application distinguishes offline-capable, offline-readable, and online-required operations. Entry creation/editing and local history are offline-capable. Viewing cached Circle content is offline-readable. Joining a Circle by invite code and destructive account operations are online-required.

26. Conflict Resolution

Conflict policy is aggregate-specific. Entries use stable IDs and timestamps with explicit treatment of user-authored fields, remote updates, and deletion. Server timestamps establish remote ordering but do not erase local intent. Deletion uses tombstones or durable delete actions until remote confirmation prevents resurrection.

Automatic resolution is allowed only when the rule is deterministic and preserves user work. Ambiguous concurrent edits should prefer a recoverable local copy or explicit review rather than silent loss. Conflict policies are pure and unit-tested.

27. Social Feature Architecture

Purpose. Private Circle hub, membership, shared entry projections, reactions, comments, and reports.

Current implementation. The repository contains 47 Swift files under `Social` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

28. Entries Feature Architecture

Purpose. The Entries feature area.

Current implementation. The repository contains 38 Swift files under `Entries` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

29. Home Feature Architecture

Purpose. Personal library/timeline access, search, filters, recent memories, and entry actions.

Current implementation. The repository contains 32 Swift files under Home feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from SwiftData projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

30. AwardsCulture Feature Architecture

Purpose. The AwardsCulture feature area.

Current implementation. The repository contains 24 Swift files under AwardsCulture feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from SwiftData projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

31. Settings Feature Architecture

Purpose. Profile, appearance/language, legal, attribution, support, sign-out, and deletion.

Current implementation. The repository contains 23 Swift files under `Settings` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

32. Battle Feature Architecture

Purpose. Interactive decision support using candidate comparisons and persisted outcomes.

Current implementation. The repository contains 19 Swift files under `Battle` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

33. QuickAdd Feature Architecture

Purpose. Low-friction creation of past watches into the canonical Entry aggregate.

Current implementation. The repository contains 15 Swift files under `QuickAdd` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

34. Wrap Feature Architecture

Purpose. The Wrap feature area.

Current implementation. The repository contains 12 Swift files under `Wrap feature module`. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

35. Notifications Feature Architecture

Purpose. Permission education, local scheduling, deep-link routing, and cleanup.

Current implementation. The repository contains 12 Swift files under `Notifications` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

36. Watchlist Feature Architecture

Purpose. Local-first intent list synchronized by owner.

Current implementation. The repository contains 7 Swift files under `watchlist` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

37. Discover Feature Architecture

Purpose. External metadata exploration and pathways into watchlist, QuickPick, or entry creation.

Current implementation. The repository contains 6 Swift files under `Discover` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

38. Onboarding Feature Architecture

Purpose. Authentication, value introduction, initial profile/state, and optional history seeding.

Current implementation. The repository contains 6 Swift files under `Onboarding` feature module. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

39. Insights Feature Architecture

Purpose. Derived personal history summaries and taste-facing previews.

Current implementation. The repository contains 5 Swift files under `Insights feature module`. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

40. Search Feature Architecture

Purpose. The Search feature area.

Current implementation. The repository contains 3 Swift files under `Search feature module`. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

41. AppShell Feature Architecture

Purpose. The AppShell feature area.

Current implementation. The repository contains 2 Swift files under `AppShell feature module`. This is direct implementation evidence, but runtime completion still depends on build and TestFlight verification.

Architecture. Presentation owns screen state and navigation. Persistent reads come from `SwiftData` projections; mutations flow through repositories or focused services. Remote collaboration, when present, uses DTO/data-source boundaries and synchronization.

Data flow. User intent is validated, translated into a domain operation, committed locally when allowed, reflected immediately in UI, and queued or sent remotely according to connectivity and authorization requirements.

Dependencies. The feature may depend on domain models, repository/service seams, shared design components, and platform adapters. It must not own Firebase paths or duplicate aggregate persistence.

Extension points. Additions should introduce focused state objects or coordinators before expanding already-large views. New remote behavior requires rules, indexes, emulator tests, and migration analysis.

Future evolution. Product and experience intent may extend this area, but new behavior must preserve private-by-default semantics, calm interaction, accessibility, and Android parity at the semantic level.

47. Testing Strategy

The repository contains 9 test files, including QuickPick engine, search pipeline, pending queue, conflict policy, duplicate detection, entry filtering/drafting/validation, and UI smoke coverage. Backend package scripts define emulator tests for rules and functions.

The test pyramid prioritizes pure domain tests, repository tests with in-memory SwiftData, sync contract tests, Firebase emulator security/integration tests, focused ViewModel tests, and a small number of stable end-to-end UI journeys. Every production defect should add the lowest-level regression test that proves the violated invariant.

48. Unit and UI Testing

Unit tests must be deterministic: inject clocks, UUID sources, connectivity, and external clients where behavior depends on them. Recommendation tests use fixed candidate histories and assert both selection and explanation. Sync tests assert queue transitions, retries, and idempotency.

UI tests cover launch gates, sign-in/onboarding, Quick Add, entry creation/edit, offline pending indicators, Circle access, and account deletion confirmation. Accessibility identifiers are stable contracts for critical controls, not visual implementation details.

49. Performance and Accessibility Engineering

Performance work starts with ownership-correct SwiftData predicates, bounded Firestore queries, image sizing/caching, task cancellation, and avoiding repeated derived computation in view bodies. Broad @Query fetches filtered in memory are migration targets. N+1 Circle membership pulls require batching or query redesign before scale.

Accessibility follows the Experience Design System: semantic labels and hints, Dynamic Type without clipping, sufficient contrast, reduce-motion behavior, keyboard/switch navigation where relevant, meaningful focus order, non-color state communication, and localized accessibility copy. Accessibility failures block release for core journeys.

54. Android Parity

Android shares product semantics, backend contracts, privacy rules, IDs, timestamps, and synchronization behavior; it does not share UI code. Kotlin, Jetpack Compose, Room, WorkManager, Hilt, and Firebase Android SDK are the expected platform choices.

Room corresponds to SwiftData as local authority. Compose corresponds to SwiftUI as declarative presentation. WorkManager handles durable background sync where iOS uses platform-appropriate scheduling and foreground/session triggers. DTO field names and enum wire values must be contract-tested across platforms. Parity means equivalent user outcome and data semantics, not pixel identity.

55. Future Architecture

Future capabilities are admitted through bounded architecture:

- Widgets consume read-only, privacy-minimized snapshots in an App Group; no sensitive journal text by default.
- Apple Watch supports lightweight capture and decision moments with explicit sync semantics.
- Vision Pro is a separate spatial experience, not a port of phone navigation.
- AI layer is opt-in, privacy-reviewed, and subordinate to deterministic product semantics.
- Recommendation Engine / Taste DNA evolves from explainable local signals toward versioned models with evaluation and rollback.
- Cinema Intelligence enriches entries with venue/context signals and personal insights before any aggregate anonymous product.
- Platform expansion reuses backend contracts and domain vocabulary while preserving native experience.

56. Architecture Governance

Architecture changes use a lightweight RFC/ADR process. A proposal states the problem, invariants, alternatives, privacy impact, data migration, platform parity, rollout, rollback, and documentation changes. ADRs are immutable after acceptance except for status and supersession links.

AI coding assistants must be given this book, the relevant product/design sections, and current code context. They may generate implementation, tests, and migration drafts, but may not invent schema fields, bypass repositories, weaken rules, or claim runtime verification they did not perform.